



Golang Gin Backend Pertemuan 10

Membahas routing, parameter URL, JSON binding, middleware, route group, dan pengenalan GORM beserta latihan praktik intermediate.



Juara Coding

June 2026



Tujuan Pembelajaran

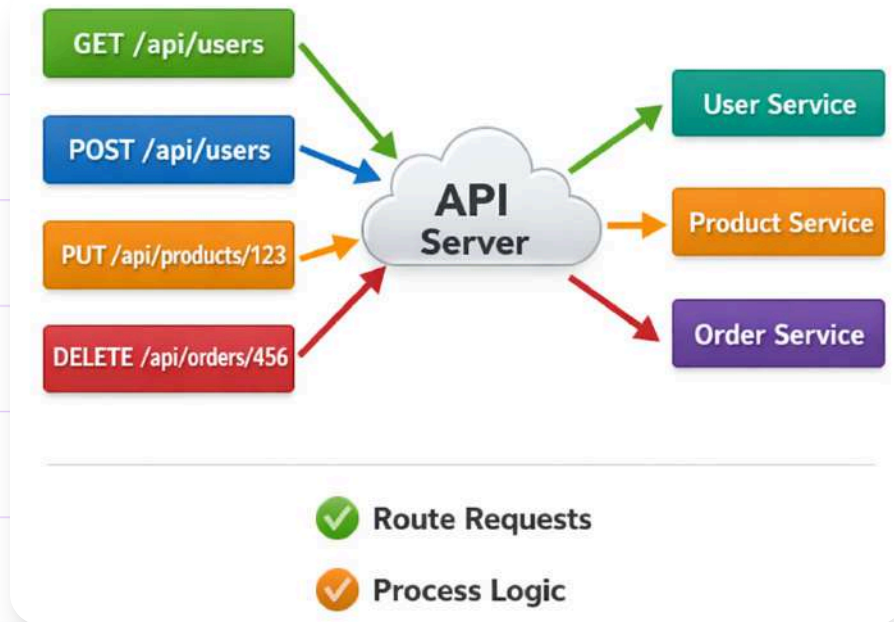
Peserta mampu membuat route Gin, membaca parameter URL dan query, menerima JSON, menambah middleware, grouping route, dan dasar GORM.



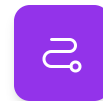
Pengenalan Gin dalam Backend Go

Gin adalah framework HTTP Go yang ringan dan cepat untuk REST API modern.

Konsep Dasar Routing



Routing memetakan HTTP method dan URL ke handler tertentu agar backend lebih rapi, mudah diuji, dan mudah dikembangkan.



Pemetaan Request

Routing menghubungkan method HTTP dan URL ke handler yang sesuai di aplikasi.



Method Gin

Gin menyediakan route dengan method seperti GET, POST, PUT, dan DELETE.



Struktur Rapi

Susunan routing yang baik memudahkan pemeliharaan, pengujian, dan pengembangan backend.

Contoh Routing Sederhana

```
// gin.Default()

// User routes
r.GET("/users", getUsers)
r.POST("/users", createUser)
r.GET("/users/:id", getUserID)
r.PUT("/users/:id", updateUser)
r.DELETE("/users/:id", deleteUser)

// Article routes
r.GET("/articles", getArticles)
r.POST("/articles", createArticle)
r.GET("/articles/:id", getArticleByID)
r.PUT("/articles/:id", updateArticle)
r.DELETE("/articles/:id", deleteArticle)
```

01 Inisialisasi Router

Router dibuat dengan `gin.Default()` untuk menyiapkan middleware bawaan dan menangani request.

03 Route dengan Parameter

Route seperti `PUT /users/:id` dan `DELETE /users/:id` memakai parameter path untuk identifikasi data.

02 Mapping Endpoint

Endpoint dipetakan berdasarkan method dan path, seperti `GET /ping` dan `POST /users`.

Memahami Path Parameter

GET /api/users/{id}

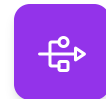
ID: None

Path parameter mengambil nilai dinamis dari URL, seperti ID resource, untuk endpoint detail, update, dan delete.



Nilai dinamis URL

Digunakan untuk membaca bagian URL yang berubah, seperti ID pada resource tertentu.



Umum pada endpoint CRUD

Sering dipakai pada endpoint detail, update, atau delete agar resource bisa diakses spesifik.



Akses lewat context Gin

Gin mengambil path parameter langsung dari context sehingga handler tetap ringkas dan jelas.

Contoh Path Parameter

```
package main

import {
    "github.com/gin-gonic/gin"
    "net/http"
}

func getUser(c *gin.Context) {
    id = c.Param("id")
    c.JSON(http.StatusOK, gin.H{
        "message": "User ID: " + id,
    },
),

func main() {
    r = gin.Default()
    r.GET("/user/:id", getUser)
    r.Run(":8080")
}
```

01 Definisi Route

Route GET /products/:id menggunakan path parameter id yang diambil dari URL request.

02 Ambil Nilai Parameter

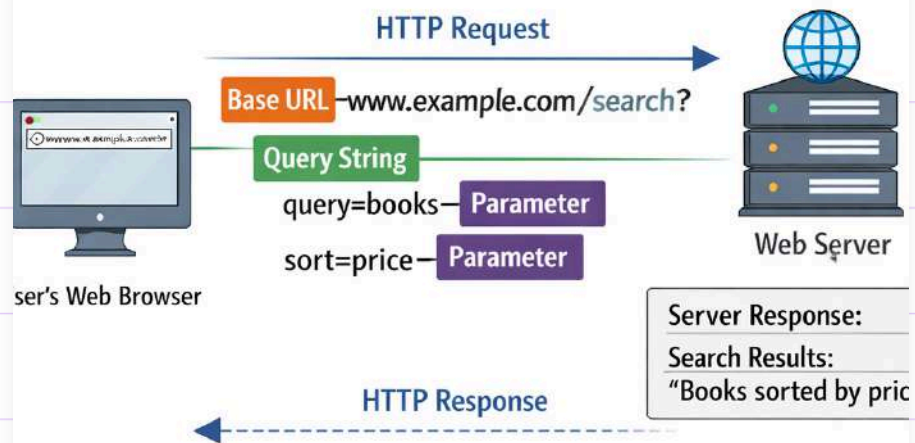
Handler memanggil `c.Param("id")` untuk membaca nilai id lalu menyimpannya ke variabel `id`.

03 Contoh Hasil Akses

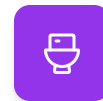
Saat client membuka /products/12, handler menerima nilai id sebagai 12 dan mengembalikannya dalam respons JSON.

Memahami Query Parameter

URL Query Parameters in Web Request



Parameter query menambahkan data opsional pada URL untuk pencarian, filter, sorting, dan pagination setelah tanda tanya.



Posisi di URL

Query parameter ditulis setelah tanda tanya dan berisi pasangan key dan value.



Data Tambahan

Digunakan untuk mengirim data seperti pencarian, filter, sorting, atau pagination.



Bersifat Opsional

Tidak mengubah path utama, sehingga cocok untuk kebutuhan request yang opsional.

Query Parameter di Gin

```
}  
func main() {  
    r = gin.Default()  
    r.GET("/search", func(c *gin.Context) {  
        term := c.DefaultQuery("term", Query("page", "1"))  
        if err := nil {  
            page = 1  
            c.JSON(http.StatusOK, gin.H{  
                "search_term": term,  
                "page": page,  
            })  
        }  
    })  
}
```

01 Mengambil query parameter

Gunakan `c.Query("keyword")` untuk membaca nilai parameter keyword dari URL request.

02 Memberi nilai default

`c.DefaultQuery("page", "1")` akan memakai nilai 1 jika parameter page tidak dikirim client.

03 Mengirim respons JSON

Handler mengembalikan data keyword dan page dalam format JSON agar mudah diuji dan dipakai.

Path vs Query Parameter

Aspect	Path Parameter	Query Parameter
Purpose	Specific resource ID	Filter or extra option
Example	/users/5	/users?role=admin
Common Use	Resource access	Search and pagination
Design Impact	Clear resource path	Flexible request control

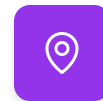
Path parameter digunakan untuk identitas resource spesifik, sedangkan query parameter dipakai untuk filter atau opsi tambahan agar desain API lebih jelas dan konsisten.

JSON Binding pada Request Body

```
// RequestBody represents the JSON request structure
import "time"

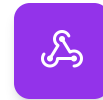
type RequestBody struct {
    UserID    int64    "json:\"user_id\""
    Name      string   "json:\"name\""
    Email     string   "json:\"email\""
    IsActive  bool     "json:\"is_active\""
    Roles     []string "json:\"roles\""
    CreatedAt time.Time "json:\"created_at\""
    Preferences map[string]interface{} "json:\"preferences\""
    Tags       []string "json:\"tags\""
    Metadata   map[string]interface{} "json:\"metadata\""
}
```

JSON binding membaca body request JSON lalu memetakannya ke struct Go agar input mudah divalidasi dan diproses.



Pemetaan ke Struct Go

Body request JSON diubah menjadi struct Go sehingga field input lebih mudah diakses.



Penting untuk Create dan Update

Fitur ini umum dipakai pada endpoint create dan update yang menerima data dari client.



Validasi Lebih Mudah

Binding membantu memeriksa format input sebelum data diproses lebih lanjut secara terstruktur.

Contoh JSON Binding

```
type User struct {
    Name string "json:\"name\""
    Age  int   "json:\"age\""
    Email string "json:\"email\""
}

// JSON binding handler
func CreateUser(c ** gin.Context) {
    var user User
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{
            "error": err.Error(),
        })
        return
    }
    c.JSON(http.StatusOK, gin.H{
        "message": "User created successfully!",
        "data": user,
    })
}
```

01 Struct Input

UserInput memiliki field *Name* dan *Email* dengan tag JSON untuk pemetaan data request.

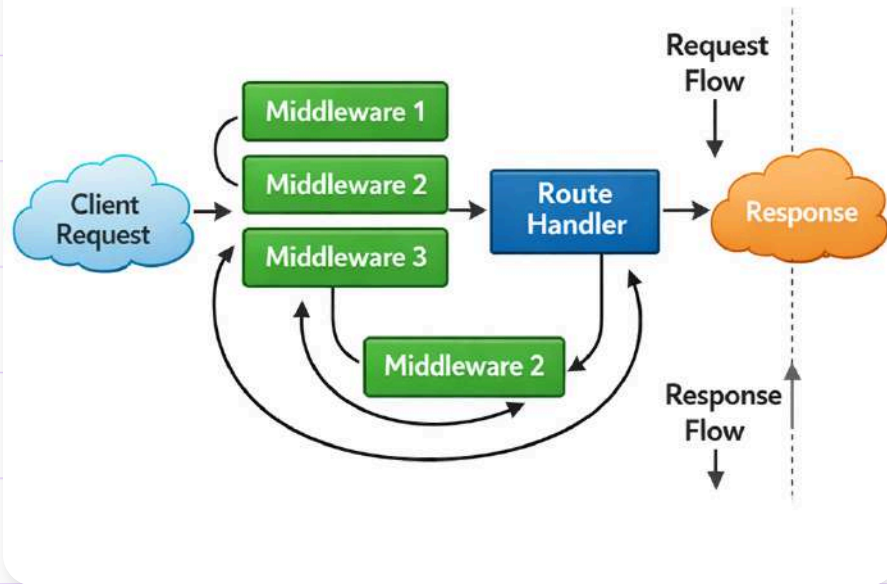
02 Binding JSON

Handler memakai *ShouldBindJSON* untuk membaca body JSON dan mengisi struct secara otomatis.

03 Respons Handler

Jika binding gagal, API mengirim error *400*; jika berhasil, data input dikembalikan dengan status *200*.

Middleware dalam Gin



Middleware berjalan sebelum atau sesudah handler untuk menangani logika umum seperti logging dan autentikasi.



Sebelum atau sesudah handler

Middleware membungkus handler utama untuk memproses request di awal atau setelah respons dibuat.



Fungsi umum terpisah

Cocok untuk logging, autentikasi, validasi header, dan pencatatan request secara konsisten.



Mengurangi pengulangan kode

Logika bersama dipisahkan agar tidak perlu ditulis ulang pada setiap endpoint aplikasi.

Contoh Middleware Sederhana

```
func AuthMiddleware(next http.Handler) http.HandlerFunc {  
  
    authHeader := r.Header.Get("Authorization")  
    if authHeader == "" {  
        http.Error(w, "Missing Authorization Header", // Authorizati  
                                                             // 401 Unauth  
        return  
    }  
    next.ServeHTTP(w, r)  
  
    return AuthMiddleware  
}
```

01 Fungsi Middleware

Middleware digunakan untuk memeriksa header *Authorization* sebelum request diproses lebih lanjut.

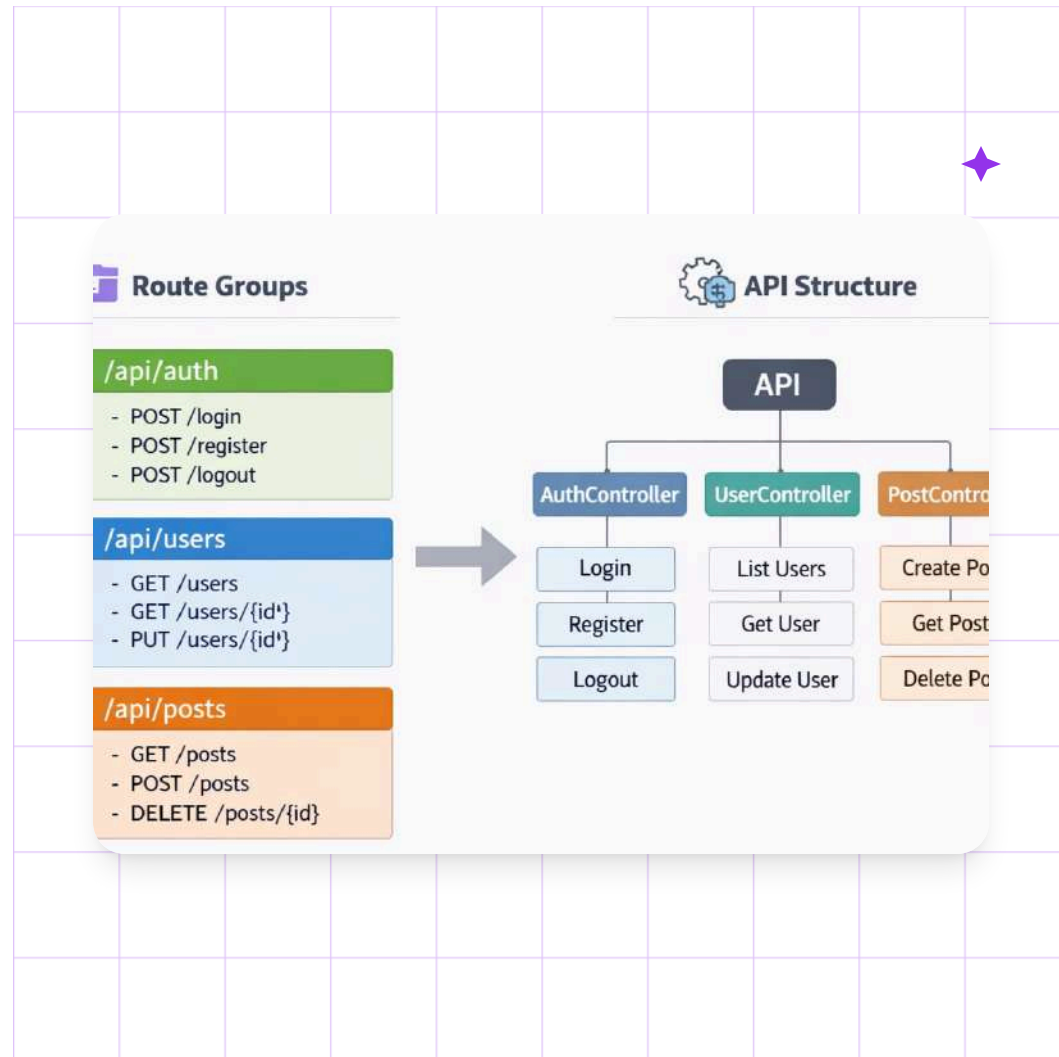
02 Validasi Token

Jika header tidak ada atau kosong, server mengembalikan respons *401 unauthorized* dan menghentikan proses.

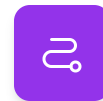
03 Lanjut ke Handler

Jika token tersedia, middleware memanggil *c.Next()* agar request diteruskan ke handler berikutnya.

Menggunakan Route Group



Route group mengelompokkan endpoint dengan prefix atau middleware yang sama agar struktur route lebih modular dan mudah dikelola.



Prefix Bersama

Beberapa endpoint dapat memakai prefix yang sama sehingga definisi route lebih ringkas.



Middleware Terpusat

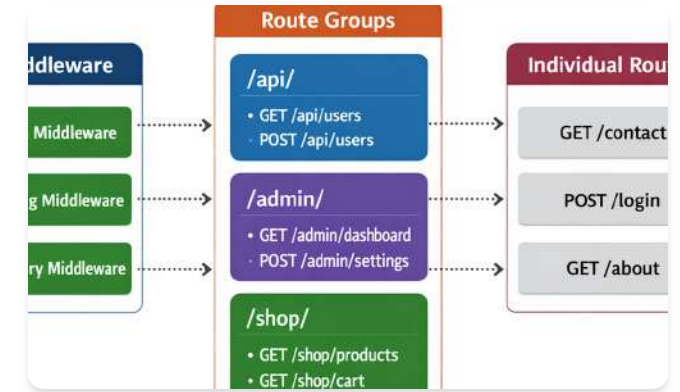
Middleware dapat diterapkan ke satu grup agar pengelolaan akses lebih efisien dan konsisten.



Struktur Modular

Route lebih mudah dibaca dan dirawat saat aplikasi berkembang menjadi lebih besar.

Contoh Route Group



01 Prefix bersama

Group dipakai untuk memberi prefix seperti */api* pada beberapa route sekaligus.

03 Struktur lebih rapi

Pengelompokan route membantu membuat definisi endpoint lebih teratur dan mudah dipelihara.

02 Middleware khusus

Route dalam group *admin* memakai *AuthMiddleware()* agar akses ke endpoint terkait lebih terkontrol.



Pengenalan Awal GORM

GORM adalah ORM untuk Go yang memudahkan interaksi database lewat struct dan method Go tanpa SQL mentah untuk operasi dasar.

Model dan Koneksi GORM

```
"database/sql"
"github.com/lib/pq"

func main() {

    dsn := "host=localhost user=dbuser password=dbpass dbname=mydb"

    db, err := sql.Open("postgres", dsn)
    if err != nil {
        fmt.Println("Failed to connect to database"; err)
    } return
    defer db.Close()
    err = db.Ping()
    if err != nil {
        fmt.Println("Database ping failed"; err)
    } return
}
```

01 Model Product Sederhana

Struct *Product* memiliki field *ID*, *Name*, dan *Price* sebagai representasi tabel data produk.

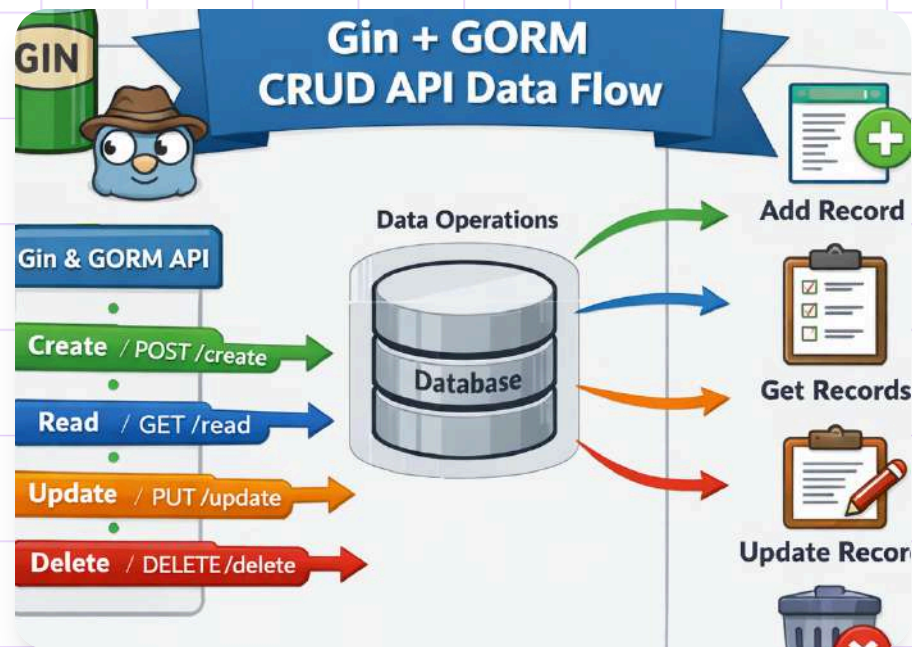
02 Koneksi ke SQLite

GORM dibuka dengan *sqlite.Open("app.db")* dan konfigurasi dasar untuk menghubungkan aplikasi ke database.

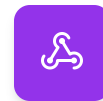
03 Migrasi Awal Otomatis

Pemanggilan *DB.AutoMigrate(&Product{})* membuat atau menyesuaikan tabel berdasarkan model yang telah didefinisikan.

Integrasi Gin dan GORM

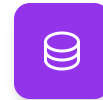


Gin menangani HTTP request, lalu GORM mengelola penyimpanan data dari JSON ke database untuk API CRUD yang terstruktur.



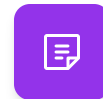
HTTP Request dengan Gin

Client mengirim JSON, lalu Gin melakukan binding request ke struct sesuai format data.



Persistensi dengan GORM

Setelah data terikat ke struct, GORM menyimpan dan mengelola data di database.



Alur CRUD Terstruktur

Kombinasi Gin dan GORM memudahkan pembuatan API CRUD yang rapi dan umum digunakan.

Latihan Praktik Intermediate 1



01 Detail buku by path parameter

Buat endpoint **GET /books/:id** untuk mengambil detail buku berdasarkan ID yang dikirim melalui path parameter.

02 Filter daftar dengan query

Sediakan **GET /books?author=...&page=...** agar daftar buku bisa difilter penulis dan dipaginasi dengan query parameter.

03 Middleware Authorization

Tambahkan middleware sederhana untuk memeriksa header **Authorization** pada seluruh route **/books** sebelum request diproses.